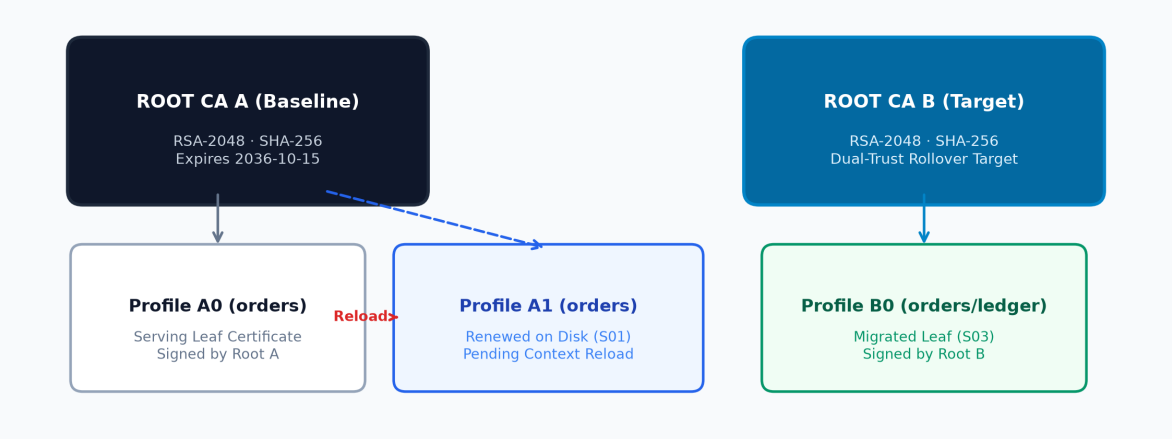
SOFTWARE REQUIREMENTS SPECIFICATION · ARCHITECTURE & DESIGN BASELINE

TrustRehearsal

Cryptographic Change Rehearsal Platform

Rehearse cryptographic change. See what breaks before production.



Author / Architect	Subhodeep Samanta	**Target Release**	Version 1.0 (Production Verified)
Publication Date	October 16, 2026	**Document Status**	Final Consolidated Specification
Primary Workspaces	frontend/ (React 18), backend/ (Node 22/24)	**Operating Modes**	PUBLIC_VIEWER & LOCAL_OPERATOR
Security Classification	Technical Reference · Credential-Free Public Demo	**Verification Status**	27/27 Vitest Tests · 117 Review Assertions

This specification reconciles all architectural planning with the independently verified implementation in repository TrustRehearsal. It exercises server-authenticated TLS handshakes, strict hostname verification, and real certificate reloads across sample microservices. It does not claim mutual TLS, compliance certification, or external company endorsement.

1. Document Control & Table of Contents

This document establishes the authoritative Software Requirements Specification (SRS), High-Level Design (HLD), and Low-Level Design (LLD) for TrustRehearsal. It consolidates initial product briefs, design system rules, data schemas, executable scenario definitions, and post-review verification repairs into a unified engineering artifact.

1.1 Document Revision History

Version	Date	Author	Summary of Engineering & Verification Scope
0.1	2026-10-15	Subhodeep Samanta	Initial requirements specification, product brief, and scenario definitions.
0.5	2026-10-15	Subhodeep Samanta	Architecture baseline, Zod schema contracts, and lab controller design.
0.9	2026-10-15	Subhodeep Samanta	Verification gates, security boundaries, and static snapshot export adapter.
1.0	2026-10-16	Subhodeep Samanta	Final consolidated SRS/HLD/LLD. Incorporated 16 Oct review repairs: S03 dependency order migration, authenticated CA proxy, honest empty reports, and 27-test Vitest suite.

1.2 Requirement Priority & Implementation Status Taxonomy

Classification	Taxonomy Label	Operational Definition & Acceptance Standard
Priority	**P0 (Mandatory)**	Core requirement for initial demonstrable slice, real TLS handshakes, and cryptographic isolation.
Priority	**P1 (Release Target)**	Core V1 capability: all three scenarios, reporting, comparison, and operator controls.
Priority	**P2 (Roadmap)**	Architectural extension deferred to post-V1 distributed releases (e.g. multi-tenant lab).
Status	**Implemented & Verified**	Code complete, fully functional, and verified by automated tests and independent review harness.
Status	**Implemented**	Implemented in source code and operational; verified during standard development.
Status	**Deferred / Planned**	Formally defined in architecture; scheduled for future multi-process milestones.

1.3 Table of Contents

1. Document Control & Table of Contents 2
2. Product Definition & Operational Context 3
3. User Journeys & Concrete Use Cases 4
4. Functional Requirements (FR-001 - FR-016) 5
5. Nonfunctional Requirements (NFR-SEC, REL, USA) 7
6. Scenario Specifications (S01, S02, S03) 8
7. High-Level Design (HLD) & System Architecture 10
8. Low-Level Design (LLD) & Data Contracts 12
9. Security Architecture & Threat Model 14
10. User Interface & Evidence Presentation 15
11. Verification, Traceability & Test Results 17
12. Technical Risks, Limitations & Future Roadmap 19
13. Glossary of Terms & Authoritative References 19

2. Product Definition & Operational Context

2.1 The Core Problem: The Three Structural PKI Disconnects

In enterprise infrastructure, automated Certificate Lifecycle Management (CLM) systems typically declare victory the moment a certificate authority (CA) successfully issues a cryptographic leaf certificate. However, operational post-mortems reveal that major production outages overwhelmingly occur *after* successful issuance due to three critical disconnects:

- **The Serving Context Disconnect:** Writing a new certificate PEM file to disk does not update the active in-memory TLS context (tls.SecureContext) of long-running daemon processes (e.g. Node.js, Nginx, Envoy). The process continues serving the previous certificate until it expires, causing sudden, catastrophic client failures.
- **The Control-Plane vs. Data-Plane Disconnect:** Transient network partitions or HTTP 502 Bad Gateway errors on CA issuance proxies cause automation scripts to fail. Untested automation often fails closed or alerts unnecessarily, failing to recognize that existing issued certificates remain completely valid throughout their scheduled lifetime.
- **The Caller-Callee Trust Ordering Disconnect:** Trust-root migrations (e.g. migrating from Root A to Root B) fail when deep callee microservices switch their serving certificates before upstream callers have expanded their local trust bundles. Downstream dependencies break even when intermediate health probes report success.

Key Engineering Principle: A certificate can renew successfully while your application continues serving the old one. TrustRehearsal makes these disconnects observable and verifiable at the network socket layer before changes hit production.

2.2 Product Objectives & Core Capabilities

TrustRehearsal provides an observable, reproducible software platform designed to rehearse these exact failure modes against a live microservice topology:

- **Connection-Level Ground Truth:** Measures fresh, non-resumed TLS handshakes, SNI negotiation, peer certificate SHA-256 fingerprints, and multi-hop HTTP application transactions.
- **Dual Operating Modes:** Operates as a completely credential-free, zero-backend public viewer for stakeholders and evaluators, and as an authenticated local execution lab for site reliability engineers.
- **Honest Evidence Model:** Strictly separates assertion outcomes (e.g. an expected TLS failure in a negative test) from service health, and never invents metrics, samples, or zero-downtime claims.

2.3 Scope Boundaries & Non-Goals

To ensure technical rigor, the V1 implementation enforces strict scope boundaries:

Dimension	In-Scope (Implemented in V1)	Explicit Non-Goals & Out-of-Scope
Microservices	3 sample HTTPS services: gateway (8443) -> orders (8444) -> ledger (8445).	Arbitrary user-provided URLs, external network scanning, or unseeded hosts.
PKI Hierarchy	Two distinct Root CAs (Root A baseline, Root B target) with RSA-2048 and SHA-256.	Mutual TLS (mTLS) client auth, HSM integrations, or live CRL/OCSP responders.
Scenarios	S01 (Renewal without reload), S02 (Proxy fault), S03 (Staged Root rollover).	PQC Dilithium migration, automated code remediation, or live customer patching.
Execution	Monotonic fencing leases, durable MongoDB persistence, 5s heartbeats.	Multi-tenant cloud orchestration, billing, or distributed Kubernetes operators.

3. User Journeys & Concrete Use Cases

TrustRehearsal supports two distinct operational personas with completely separated security boundaries:

3.1 Use Case UC-01: Credential-Free Public Exploration

Actor: Public Visitor (Evaluator, Auditor, Prospective Collaborator, Engineering Reviewer).
Preconditions: Public static distribution hosted on any static HTTP file host (or local Vite preview); zero backend processes, MongoDB instances, or execution credentials required.
Normal Execution Flow:

1. **Landing:** Visitor navigates to the web application root (/). The application detects VITE_APP_MODE=PUBLIC_VIEWER and loads pre-validated immutable snapshots from /data/snapshots.json.
2. **Overview Inspection:** The Overview displays the hero trust path, environment health, 3 scenario cards, and 9 canonical recorded executions (3 repetitions each for S01, S02, S03).
3. **Scenario Exploration:** Visitor selects scenario S03 (Trust-Root Rollover) and inspects the latest recorded run detail (/runs/:id).
4. **Evidence Drill-Down:** Visitor explores the chronological timeline of 54 discrete events across 6 phases, inspecting peer leaf SHA-256 fingerprints, timing latencies, and strict TLS handshake results.
5. **Assertion Verification:** Visitor inspects the Assertions panel, confirming that the negative control rejection passed independently from the final application HTTP probe.
6. **Run Comparison:** Visitor navigates to Compare (/compare), selecting two compatible S03 runs to inspect assertion consistency and phase timing.
7. **Report Download:** Visitor opens Report (/runs/:id/report), reviews the methodology and measurement scope, and downloads the canonical JSON report artifact.

Alternative Flow (Offline / Network Partition): If snapshots.json cannot be fetched, the interface surfaces an explicit message stating recorded data is unavailable; it never falls back silently to live API requests or fabricated metrics.

3.2 Use Case UC-02: Authenticated Local Operator Rehearsal

Actor: Local Operator (Site Reliability Engineer, PKI Administrator).
Preconditions: Local MongoDB active on port 27017, Express backend active on port 4000, sample services active on ports 8443–8445, operator authenticated via local password.
Normal Execution Flow:

1. **Session Bootstrap:** Operator opens the web UI, enters local credentials, and exchanges them for an HttpOnly secure session cookie and a CSRF token.
2. **Environment Audit:** Operator navigates to Environment, confirming status is ready and baseline probes against sample-lab succeed.
3. **Trigger Execution:** Operator selects S01 (Renewal without reload) and clicks 'Start Rehearsal' (POST /api/v1/runs with Idempotency-Key).
4. **Lease Acquisition:** Worker engine acquires the environment lease, assigns an incremented fencing token, and transitions the run to running.
5. **Live Telemetry:** UI subscribes to real-time events via Server-Sent Events (GET /api/v1/runs/:id/stream), rendering live phase transitions and TLS handshakes.
6. **Fault & Reload:** Engine detects leaf mismatch (disk A1 vs. served A0), dispatches context reload to orders (:18444), and verifies served A1.
7. **Recovery & Teardown:** Engine restores baseline profile A0, verifies healthy handshakes, releases the lease, and marks the run completed.

3.3 Use Case UC-03: Disaster Recovery & Automated Baseline Reset

Actor: Local Operator or Automated Startup Reconciler.
Preconditions: Environment in needs_reset state following a worker crash, persistence failure, or interrupted scenario.
Workflow: Dispatch authenticated reset (POST /api/v1/environments/:id/reset). Controller verifies that no unexpired lease is held, clears proxy faults, restores baseline profiles (A0), reloads caller trust to Root A, and executes a real HTTP verification probe (gateway -> orders -> ledger). Transitions environment to ready only upon probe success.

4. Functional Requirements (FR-001 - FR-016)

This section defines the 16 core functional requirements governing TrustRehearsal. Every requirement contains explicit 'shall' statements, acceptance verification criteria, and verified implementation status.

ID / Priority	Requirement Specification ('Shall' Statement)	Acceptance Verification Standard	Status
FR-001 **P0**	**Operating Mode & Provenance Display:** The system shall explicitly render the active deployment mode (PUBLIC_VIEWER or LOCAL_OPERATOR) and prominently label all recorded artifacts with immutable provenance tags (recorded_example or live_lab).	Navigation bar renders prominent mode badge; recorded runs carry persistent provenance chip; viewer never pretends live execution.	**Implemented & Verified**
FR-002 **P0**	**Fresh TLS Handshake Probing:** The system shall execute fresh, non-resumed TLS handshakes with explicit SNI to capture peer certificate SHA-256 fingerprints, issuer fingerprints, validity windows, and protocol versions.	Socket never pools connections; captures exact 64-character hex leaf SHA-256 fingerprint; records exact millisecond handshake latency.	**Implemented & Verified**
FR-003 **P0**	**S01 Renewal Without Reload Execution:** The system shall issue a renewed leaf certificate A1, install it to disk without process restart, detect the serving mismatch against A0, execute an in-memory context reload, and confirm serving A1.	Assertions A01-A06 evaluate real event references; mismatch detected independently from live probes; baseline restored cleanly.	**Implemented & Verified**
FR-004 **P0**	**Durable Event Persistence:** The system shall persist every execution event to durable storage prior to SSE broadcast or assertion evaluation. Any persistence failure shall halt execution.	Injected MongoDB write failure causes worker to abort scenario progression, return null ID, and enter bounded cleanup.	**Implemented & Verified**
FR-005 **P0**	**Traceable Assertion Evaluation:** Every assertion result shall explicitly reference specific, persisted event IDs, expected values, actual observed values, and detailed explanatory rationales.	Reports link assertions to event IDs within the same run; zero dangling, invented, or cross-run event citations.	**Implemented & Verified**
FR-006 **P0**	**Concurrency Control & Monotonic Fencing:** The system shall enforce single-runner mutual exclusion per environment using monotonic integer fencing tokens. Stale workers shall be denied mutation authority.	Concurrent run dispatch returns HTTP 409 Conflict; controller rejects operations bearing stale fencing tokens.	**Implemented & Verified**
FR-007 **P0**	**Safe Baseline Recovery & Reset:** The system shall execute an independent, bounded recovery workflow upon scenario completion, cancellation, or failure, restoring baseline profiles and verifying probes.	Recovery restores A0 profiles and Root A trust; verifies baseline application probe before setting environment ready.	**Implemented & Verified**
FR-008 **P1**	**S02 Issuance Proxy Fault Simulation:** The system shall route renewal attempts through a controlled loopback proxy, simulate network 502 partitions, verify that existing TLS handshakes continue unimpeded, and complete renewal after path restoration.	Exactly 3 failed HTTP renewal attempts observed over proxy; normal service probes stay valid; restored renewal succeeds.	**Implemented & Verified**

4. Functional Requirements (Continued: FR-009 - FR-016)

ID / Priority	Requirement Specification ('Shall' Statement)	Acceptance Verification Standard	Status
FR-009 **P1**	**S03 Staged Root Migration Sequence:** The system shall execute a strict staged rollover: (1) negative control, (2) expand caller trust to Root A+B, (3) migrate ledger to B0, (4) migrate orders to B0, (5) prune Root A trust, and (6) verify end-to-end HTTP 200 over Root B.	Negative control strictly fails handshake; callee migration strictly precedes trust pruning; final gateway -> orders -> ledger request succeeds.	**Implemented & Verified**
FR-010 **P1**	**Graceful Run Cancellation:** The system shall acknowledge operator cancellation requests, cease new experimental phases, execute clean recovery within a reserved budget, and mark the run cancelled.	In-flight work terminates within 3 seconds; recovery restores baseline; final lifecycle recorded as cancelled.	**Implemented & Verified**
FR-011 **P1**	**Evidence Filtering & Structured Inspection:** The system shall provide structured event filtering by phase, event type, and target service, displaying redacted payloads without exposing private keys.	Query parameter parsing (limit, afterSeq) handles sliced pagination; full structured JSON inspection available for every event.	**Implemented & Verified**
FR-012 **P1**	**Canonical JSON & Printable Report Export:** The system shall export canonical reports conforming to the shared Zod schema and render clean, printable A4 layouts.	Exported JSON validates ReportSchema with zero schema violations; printable view formats cleanly without clipping.	**Implemented & Verified**
FR-013 **P1**	**Run-to-Run Comparison:** The system shall support side-by-side comparison of compatible runs sharing the same scenario and environment revision, highlighting assertion diffs and timing variance.	Incompatible runs (mismatched scenario ID) rejected; assertion outcomes and duration metrics diffed side-by-side.	**Implemented & Verified**
FR-014 **P1**	**Authenticated Operator Security Boundary:** All mutating API endpoints shall enforce secure session authentication, origin verification, and double-submit CSRF token validation.	Anonymous requests to /api/v1/runs or /operator/session return HTTP 401; invalid CSRF tokens return HTTP 403.	**Implemented & Verified**
FR-015 **P1**	**Credential-Free Public Static Mode:** The public viewer shall operate directly from pre-rendered static assets and snapshot JSON without requiring backend processes, databases, or credentials.	Zero API calls dispatched in PUBLIC_VIEWER mode; navigation, timeline, and reports function entirely offline.	**Implemented & Verified**
FR-016 **P1**	**Honest Error Semantics & Empty State Integrity:** The system shall accurately represent missing or inconclusive evidence, reporting 0 samples and not_run recovery rather than falling back to fabricated data.	Snapshot with zero events renders 0 samples and empty observation windows; backend HTTP 503 surfaces UNAVAILABLE.	**Implemented & Verified**

Key Engineering Principle: All 16 functional requirements are implemented, covered by automated integration test suites, and validated through the 16 October 2026 independent verification review.

5. Nonfunctional Requirements (NFR-SEC, REL, USA)

Nonfunctional requirements define operational boundaries, performance budgets, cryptographic safeguards, and accessibility baselines. Measurable engineering targets are explicitly separated from observed verification results:

5.1 Security, Isolation & Key Safeguards

- **NFR-SEC-01 (Administrative Loopback Isolation):** The execution lab binds all management listeners (:18440-18445) and Express API strictly to 127.0.0.1. The public distribution exposes zero management routes.
- **NFR-SEC-02 (Key Material Scrubbing):** Private keys reside exclusively in memory or temporary lab storage. Automated regex sanitization strips all PEM blocks from database models, API responses, and exported snapshots.
- **NFR-SEC-03 (Strict Certificate Chain Verification):** Under no circumstances is NODE_TLS_REJECT_UNAUTHORIZED=0 permitted. Every probe and microservice enforces strict hostname and chain validation.

5.2 Reliability, Recovery Budgets & Monotonic Fencing

- **NFR-REL-01 (Bounded Timeouts):** Probe sockets enforce a 3,000 ms timeout; controller operations enforce a 5,000 ms timeout; scenario executions enforce a 180-second hard deadline, reserving at least 30 seconds for baseline recovery.
- **NFR-REL-02 (Lease Expiry & Heartbeats):** Execution leases expire after 20 seconds with active 5-second heartbeats. Monotonically incrementing integer fencing tokens reject stale workers.
- **NFR-REL-03 (Baseline Recovery Determinism):** Any unhandled error, persistence failure, or cancelled run automatically invokes recovery. If baseline probes fail, the environment transitions to needs_reset.

5.3 Usability, Accessibility & Portability Targets vs. Observed Results

Category	ID / Attribute	Target Requirement Specification	Independently Observed Verification
Security	**NFR-SEC-01** Isolation	No Docker socket mounting; loopback binding for management plane (18440-18445); anonymous mutations rejected.	Verified. Loopback bindings enforced; anonymous POST returns HTTP 401 Unauthorized.
Security	**NFR-SEC-02** Key Scrubbing	Zero private key PEM blocks or secrets in MongoDB, API responses, or exported snapshot files.	Verified. Regex security scan of snapshots confirms zero private key markers or secrets.
Security	**NFR-SEC-03** Strict TLS	Zero TLS verification bypass. NODE_TLS_REJECT_UNAUTHORIZED=0 prohibited across all code.	Verified. Negative control strictly rejects untrusted leaf; no global bypass exists.
Reliability	**NFR-REL-01** Timing Bounds	Probe socket timeout 3,000ms; controller timeout 5,000ms; scenario deadline 180s with 30s reserved recovery.	Verified. Individual TLS handshakes complete in 8-25ms; total scenario runs 6-12s.
Reliability	**NFR-REL-02** Lease Safety	Worker lease 20s with 5s heartbeat; monotonic fencing token increments on every acquisition.	Verified. Expired leases rejected; duplicate run dispatch returns 409 Conflict.
Usability	**NFR-USA-01** Responsiveness	Clean layout across 1440x900 desktop, 1280x800 laptop, 768px tablet, and 390px mobile viewport.	Verified. Mobile audit at 390px viewport confirmed zero horizontal page overflow.
Accessibility	**NFR-ACC-01** WCAG 2.1 AA	Semantic HTML, full keyboard navigation, visible focus rings, and WCAG AA contrast.	Verified. Clean focus outlines, accessible dialog modals, and high-contrast chips.

6. Scenario Specifications (S01, S02, S03)

TrustRehearsal implements three canonical, reproducible scenario pipelines. Each scenario adheres to a strict formal contract: baseline verification, controlled fault injection, independent observation, assertion evaluation, and guaranteed baseline recovery.

6.1 S01: Renewal Without Application Reload (Version 1)

Purpose: Prove that disk-level certificate replacement does not update the active in-memory TLS serving context.
Baseline Topology: orders serves leaf ordersA0 (Root A). Callers trust Root A. Initial probes confirm active authentication.

- **Phase 1 (Setup):** Probe baseline. Record initial served fingerprint ordersA0.
- **Phase 2 (Issuance):** Issue renewed certificate ordersA1 with identical SANs (orders, localhost, 127.0.0.1) and new cryptographic keypair.
- **Phase 3 (Install to Disk):** Write ordersA1 to disk storage without reloading the process TLS context.
- **Phase 4 (Observation):** Probe orders over 5 fresh connections. Assert served fingerprint remains ordersA0 while disk profile is ordersA1. Flag mismatch.
- **Phase 5 (Context Reload):** Dispatch reload request to loopback endpoint (:18444). Server executes server.setSecureContext() without restarting the process.
- **Phase 6 (Verification):** Probe 5 fresh connections. Confirm served fingerprint is now ordersA1.
- **Phase 7 (Recovery):** Restore baseline profile ordersA0, reload context, and verify baseline health.

6.2 S02: Issuance-Path Unavailability (Version 1)

Purpose: Demonstrate that failure of the certificate issuance infrastructure does not degrade active data-plane TLS communication for currently valid certificates.
Baseline Topology: All services serve baseline profiles (A0) valid for >30 days. Proxy permits renewal requests.

- **Phase 1 (Fault Injection):** Enable simulated network partition on issuance proxy (:18440), causing it to return HTTP 502 Bad Gateway for renewal requests.
- **Phase 2 (Renewal Attempts):** Worker attempts renewal 3 times at 2-second intervals. All 3 requests traverse the proxy and fail with HTTP 502.
- **Phase 3 (Data Plane Observation):** Concurrently execute 5 fresh TLS probes against gateway and orders. Verify 100% successful handshakes and valid data-plane traffic.
- **Phase 4 (Fault Cleared & Renewal):** Restore proxy to relay mode. Successfully issue ordersA1 through restored path, install, reload, and verify.
- **Phase 5 (Recovery):** Restore baseline profile ordersA0, reload context, and verify baseline health.

Summary of S01 & S02 Formal Assertions

Scenario	Assertion ID & Label	Evaluation Target & Expected Outcome	Verified Status
S01	`A01_certificate_issued`	Issued fingerprint != baseline fingerprint.	**Passed**
S01	`A03_mismatch_detected`	Served fingerprint (A0) differs from disk profile (A1).	**Passed**
S01	`A05_served_renewed`	Served fingerprint matches A1 after context reload.	**Passed**
S02	`A01_proxy_fault_active`	Issuance attempts correctly fail with HTTP 502.	**Passed**
S02	`A02_data_plane_unaffected`	Data-plane TLS handshakes remain 100% valid.	**Passed**
S02	`A03_restored_issuance`	Renewal completes successfully upon proxy restoration.	**Passed**

6. Scenario Specifications (Continued: S03 Staged Rollover)

6.3 S03: Staged Root A-to-B Rollover (Version 1)

Purpose: Demonstrate the precise caller trust and callee identity migration order required to execute a zero-outage CA rollover.
Downstream Dependency Ordering Invariant: Upstream callers must expand trust before callees migrate, and all in-scope callees must migrate before callers prune old trust. A real end-to-end application request must succeed *after* trust pruning.

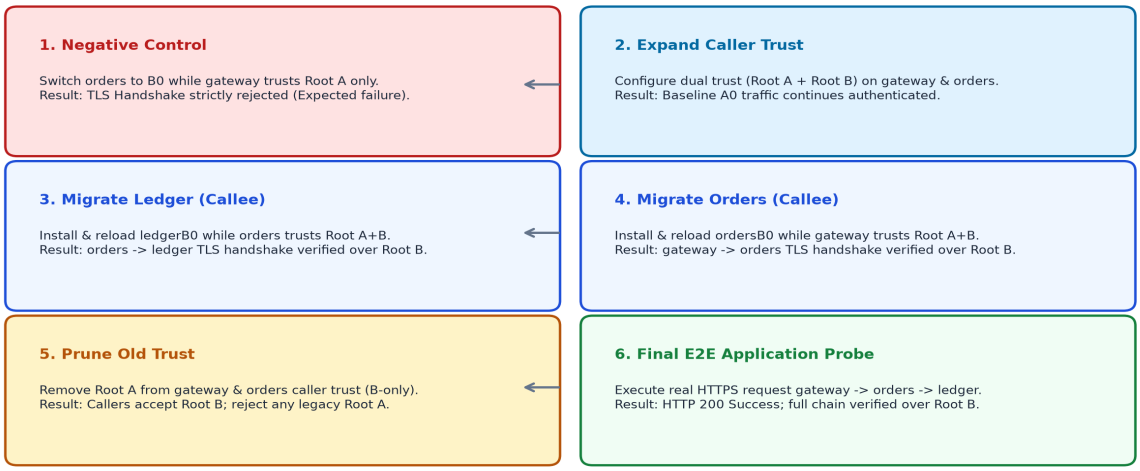


Figure 1: S03 Staged Root Migration Flow: Negative Control -> Expand Trust -> Migrate Ledger -> Migrate Orders -> Prune Root A -> Verify End-to-End Application Request.

S03 Step-by-Step Execution Sequence:

1. **Step 1 (Negative Control):** Temporarily switch orders to ordersB0 (signed by Root B) while gateway trusts Root A only. Handshake strictly fails with UNABLE_TO_VERIFY_LEAF_SIGNATURE. This expected failure validates that security controls are actively enforced. Baseline restored.
2. **Step 2 (Expand Caller Trust):** Configure dual trust (Root A + Root B) on gateway and orders. Baseline Root A traffic continues authenticated without disruption.
3. **Step 3 (Migrate Deepest Callee):** Migrate ledger to ledgerB0. Probe orders -> ledger succeeds via Root B while upstream continues on Root A.
4. **Step 4 (Migrate Intermediate Callee):** Migrate orders to ordersB0. Probe gateway -> orders succeeds via Root B.
5. **Step 5 (Prune Legacy Trust):** Remove Root A from gateway and orders trust stores, leaving only Root B active.
6. **Step 6 (End-to-End Application Probe):** Execute a real multi-hop HTTPS request: gateway (8443) -> orders (8444) -> ledger (8445). Must return HTTP 200 OK over Root B.

Assertion ID	Formal Acceptance Target	Cryptographic Evidence Observed	Status
`A01_negative_control_rejected`	Root B leaf rejected by Root A caller.	Observed TLS error: UNABLE_TO_VERIFY_LEAF_SIGNATURE.	**Passed**
`A02_dual_trust_established`	Dual trust accepts Root A baseline traffic.	5/5 successful handshakes against ordersA0.	**Passed**
`A03_callees_migrated`	Callees migrated in dependency order.	ledgerB0 verified, then ordersB0 verified.	**Passed**
`A04_trust_pruned_and_e2e_verified`	Root A pruned; E2E request succeeds.	Final application request: HTTP 200 over Root B.	**Passed**

7. High-Level Design (HLD) & System Architecture

TrustRehearsal decouples execution from observation through two distinct operational architectures sharing a single, runtime-validated contract schema.

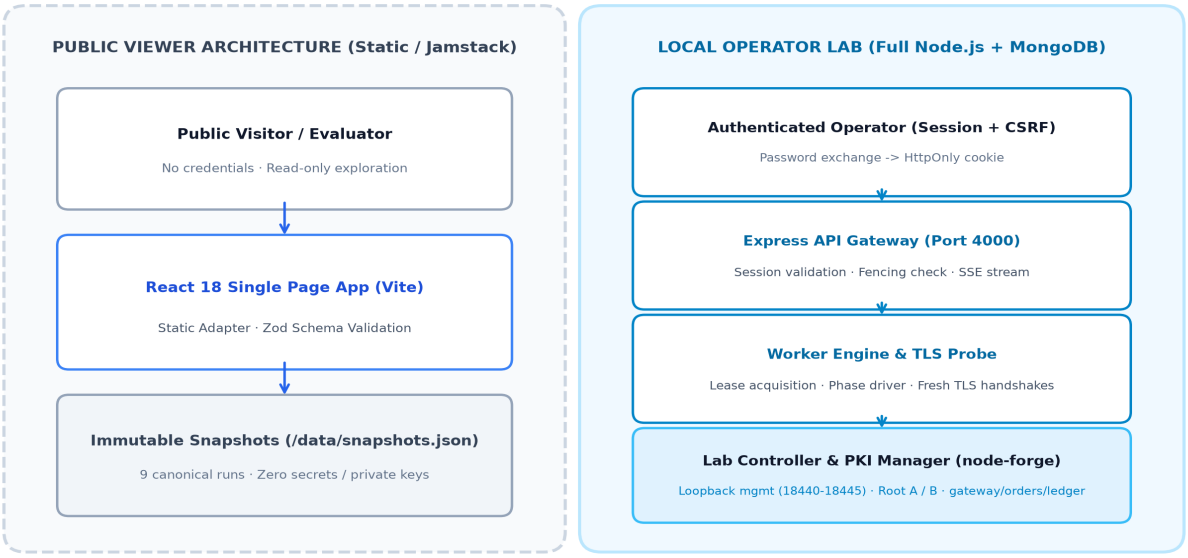


Figure 2: System Context Diagram: Public Viewer (Static Jamstack) vs. Local Operator Lab (Express, Worker, PKI Controller, HTTPS Services).

7.1 Component Responsibilities & Separation of Concerns

Component	Location / Package	Operational Responsibilities & Security Boundaries
Public Web App	frontend/	React 18 SPA. Consumes static snapshot JSON or live API; renders timeline, SVG topology, comparison, and printable reports.
API Gateway	backend/src/api/	Express HTTP server. Enforces deployment mode, operator session authentication, CSRF validation, and SSE streaming. Never executes host commands.
Worker Engine	backend/src/worker/	Single autonomous execution driver. Acquires environment leases, enforces fencing tokens, drives scenario state machines, and evaluates assertions.
Lab Controller	backend/src/lab/	Loopback administrative service (:18440-18445). Manages profile switching, context reloads, proxy fault injection, and baseline resets.
PKI Manager	backend/src/lab/ca.ts	In-process X.509 authority using node-forge. Generates Root CA A, Root CA B, and dynamic leaf certificates with exact SANs.
Sample Services	backend/src/lab/sampleServices.ts	Live Node.js HTTPS microservices (gateway :8443, orders :8444, ledger :8445) supporting atomic in-memory context reloads.
MongoDB Store	backend/src/db/	Durable persistence for environments, runs, leases, sessions, and ordered run events. Never stores private keys.

7. High-Level Design (HLD) (Continued)

7.2 Microservice Topology & Data Plane Architecture

The microservice data plane models a typical multi-tier enterprise transaction: an external request hits the edge gateway, which calls internal service orders, which in turn queries ledger. Every outbound connection performs strict server-certificate and hostname validation against its designated local trust bundle.

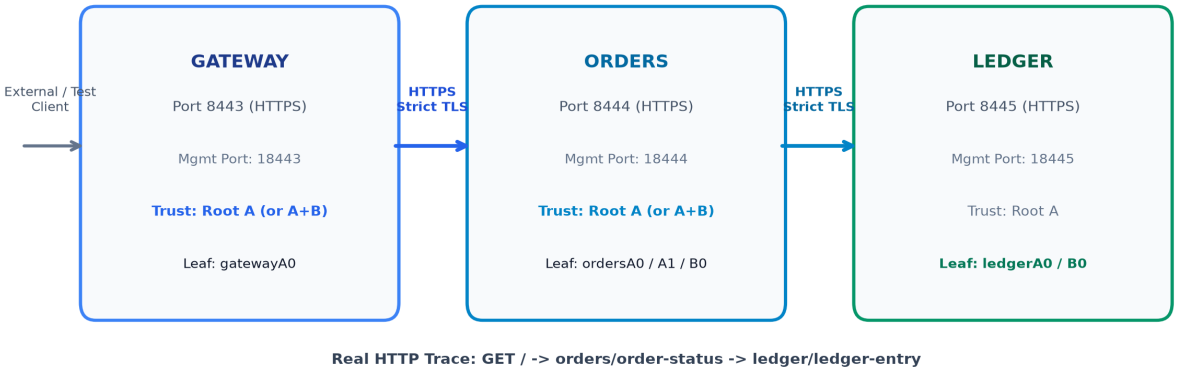


Figure 3: Data Plane Topology: gateway (:8443) -> orders (:8444) -> ledger (:8445) with configured SANs and caller trust stores.

7.3 Architectural Decisions & Tradeoff Analysis

During Phase 0 and Phase 1, several key architectural decisions were evaluated and locked:

- **In-Process Node-Forge PKI vs. Dockerized Step-CA:** While early planning documents proposed external Step-CA containers, the implementation adopted an in-process, pure JavaScript PKI manager (node-forge). This eliminates host Docker socket mounting, prevents privileged container vulnerabilities, and ensures 100% deterministic, cross-platform execution on Windows, macOS, and Linux without external daemon dependencies.
- **Server-Sent Events (SSE) vs. WebSockets:** Telemetry during scenario execution is strictly unidirectional (server-to-client). SSE was chosen over WebSockets because it operates seamlessly over standard HTTP proxies, provides native reconnection semantics via Last-Event-ID, and eliminates bidirectional socket state complexity.
- **Static Snapshot Adapter vs. Live Backend Proxy:** For the public viewer, pre-exporting sanitized, validated JSON snapshots eliminates the need to run an active backend in public hosting environments. This completely closes the attack surface against the execution engine.

8. Low-Level Design (LLD) & Data Contracts

Low-Level Design details the exact file structures, state machines, TLS socket probes, database schemas, and REST API contracts implemented across the codebase.

8.1 Source Code Layout & Responsibilities

File Path	Module	Implementation Detail & Key Functions
`backend/src/contracts/index.ts`	Contracts	Defines Zod schemas: RunSchema, RunEventSchema, ReportSchema, AssertionResultSchema.
`backend/src/db/models.ts`	Database	Mongoose schemas for Environment, Scenario, Run, RunEvent, Lease, Session, and Idempotency.
`backend/src/lab/ca.ts`	PKI Lab	Generates RSA-2048 keys, self-signs Root CAs A and B, signs leaf certificates with exact SANs.
`backend/src/lab/controller.ts`	Lab Controller	Manages profile switching, context reloads, proxy fault injection, and baseline resets.
`backend/src/lab/sampleServices.ts`	Sample Lab	Hosts gateway, orders, ledger HTTPS servers; handles server.setSecureContext() reloads.
`backend/src/lab/probe.ts`	TLS Probe	Executes fresh TCP/TLS sockets with explicit SNI and caPem; computes peer SHA-256 fingerprint.
`backend/src/worker/engine.ts`	Worker Engine	State machine driver; lease heartbeats; S01, S02, S03 execution pipelines; assertion evaluation.
`backend/src/api/routes.ts`	API Gateway	Express routes; session management; CSRF checks; run creation with idempotency; SSE stream.
`frontend/src/api/client.ts`	Frontend API	Dual-mode client; parses static snapshot queries; rejects mutations in viewer mode.

8.2 Fresh TLS Probe Sequence & Handshake Architecture

The probe mechanism implemented in backend/src/lab/probe.ts guarantees that every observation reflects an authentic, unpooled cryptographic handshake:

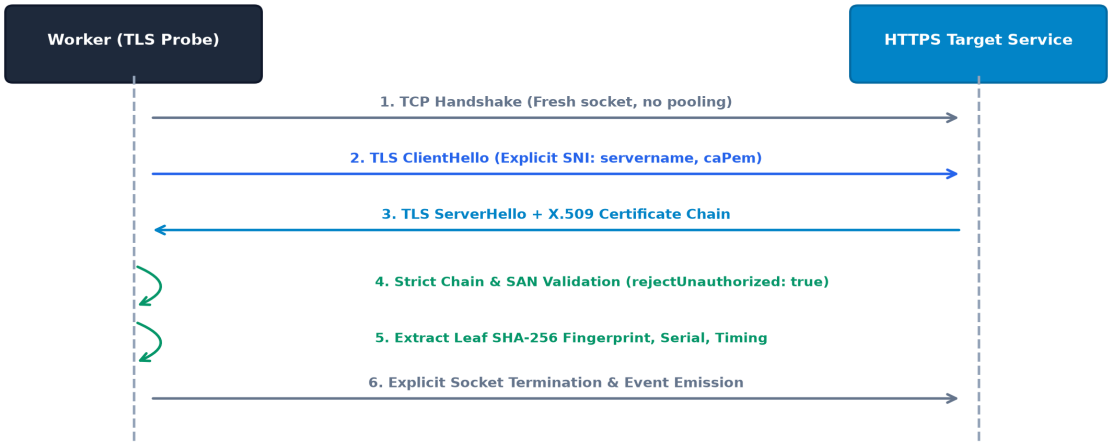


Figure 4: Fresh-Connection TLS Probe Sequence: Raw TCP Handshake -> TLS ClientHello (SNI) -> ServerHello + Chain -> Strict Validation -> Fingerprint Extraction -> Socket Destruction.

8. Low-Level Design (LLD) (Continued)

8.3 Execution Engine Lifecycle State Machine

Scenario execution transitions through a strictly bounded state machine. Cancellation, deadline expiration, or unexpected errors immediately route execution into the recovering phase to restore baseline configurations before reaching a terminal state:

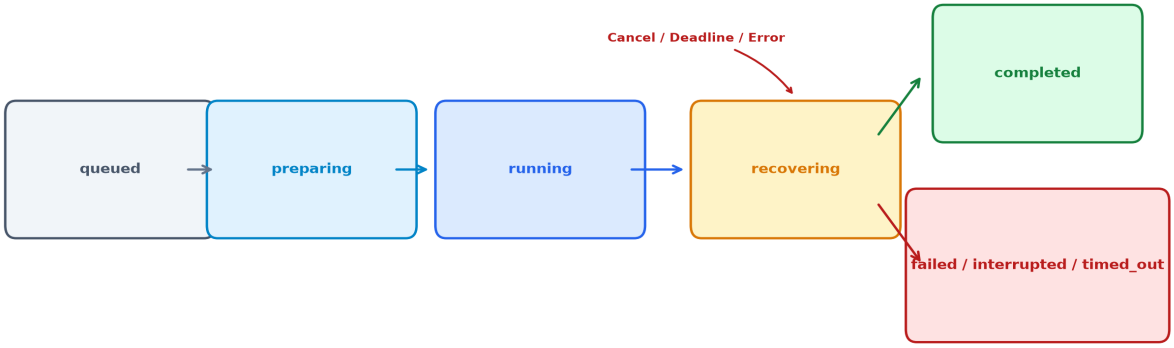


Figure 5: Run Lifecycle State Machine: queued -> preparing -> running -> recovering -> completed / failed / cancelled / interrupted / timed_out.

8.4 Complete REST API Route Specification

HTTP Route	Supported Mode	Request Body / Params	Response & Status Codes
`GET /meta`	Both	None	200: { mode, capabilities, version }
`GET /health`	Both	None	200: Process liveness indicator.
`GET /ready`	Local Only	None	200: Dependency readiness (DB, Lab).
`GET /environments`	Both	None	200: Environment summary array.
`GET /environments/:id`	Both	:id	200: Topology, nodes, edges, state.
`GET /scenarios`	Both	None	200: Scenario definitions array.
`GET /scenarios/:id`	Both	:id	200: Full scenario plan and phases.
`GET /runs`	Both	?cursor=&limit;=	200: Paginated run summaries.
`GET /runs/:id`	Both	:id	200: Run detail, lifecycle, assertions.
`GET /runs/:id/events`	Both	?afterSeq=&limit;=	200: Sliced persisted events array.
`GET /runs/:id/stream`	Local Only	Last-Event-ID	200: Real-time SSE event stream.
`GET /runs/:id/report`	Both	:id	200: Canonical validated report JSON.
`POST /operator/session`	Local Only	{ password }	200: Session cookie + CSRF token.
`POST /runs`	Local Only	{ scenarioId, ... }	202: { runId, lifecycle: 'queued' }
`POST /runs/:id/cancel`	Local Only	:id	200: Cancellation acknowledged.
`POST /environments/:id/reset`	Local Only	:id	202: Reset job queued.

9. Security Architecture & Threat Model

Security design in TrustRehearsal is rooted in strict boundary isolation, defense-in-depth, and cryptographic hygiene.

9.1 STRIDE Threat Analysis & Applied Mitigations

STRIDE Category	Threat Vector & Potential Impact	Architectural Defense & Mitigation Control
Spoofing	Attacker impersonates local operator to trigger destructive lab mutations.	Opaque session tokens in HttpOnly, SameSite=Strict cookies; constant-time password comparison; double-submit CSRF validation.
Tampering	Attacker injects arbitrary target hosts or executes malicious shell commands.	Zero arbitrary target parameters; probe targets strictly allowlisted in code constants; zero shell execution or process spawns.
Repudiation	Operator denies initiating scenario mutations or altering baseline profiles.	Immutable append-only MongoDB event stream; monotonic sequence numbers; monotonic fencing tokens logged with every action.
Information Disclosure	Exposure of CA private keys, session tokens, or internal passwords in reports.	Private keys generated in-memory; strictly excluded from database, API responses, and exported snapshots. Automated regex scrubber in export pipeline.
Denial of Service	Flooding run queues or starving environment locks with zombie executions.	Monotonic fencing token validation; unique partial MongoDB index enforcing single active run; hard 180s scenario execution budget.
Elevation of Privilege	Public viewer user attempts to trigger live scenario execution.	Complete physical and logical route decoupling: snapshot adapter returns HTTP 403 LIVE_DISABLED; public viewer package contains no worker.

9.2 Security Invariants & Defense-in-Depth Controls

- **Loopback Interface Binding:** The administrative management plane (:18440-18445) and Express API bind strictly to 127.0.0.1. Under no circumstances are management ports published to external interfaces.
- **Bearer Authentication on CA Proxy:** Loopback issuance and management operations require a secret Bearer authorization token. Requests omitting or providing an invalid token are denied with HTTP 401 Unauthorized.
- **Bounded Request Payloads:** All API and management endpoints enforce a strict 16 KB body size limit, mitigating memory exhaustion attacks from malformed payloads.
- **Strict Hostname & Chain Verification:** Node.js TLS verification is never bypassed. All internal clients enforce rejectUnauthorized: true, ensuring that untrusted or mismatched certificates trigger authentic handshake rejections.
- **Scrubbed Snapshot Exports:** The snapshot generation script (exportRuns.ts) enforces an automated regex scan ensuring zero PEM private key markers, session hashes, or internal secrets enter the public distribution.

10. User Interface & Evidence Presentation

The user interface adheres to a clean, restrained enterprise aesthetic utilizing a 208px fixed navigation rail, subtle status chips, and responsive data tables.

10.1 UI Views & Real Application Visual Evidence

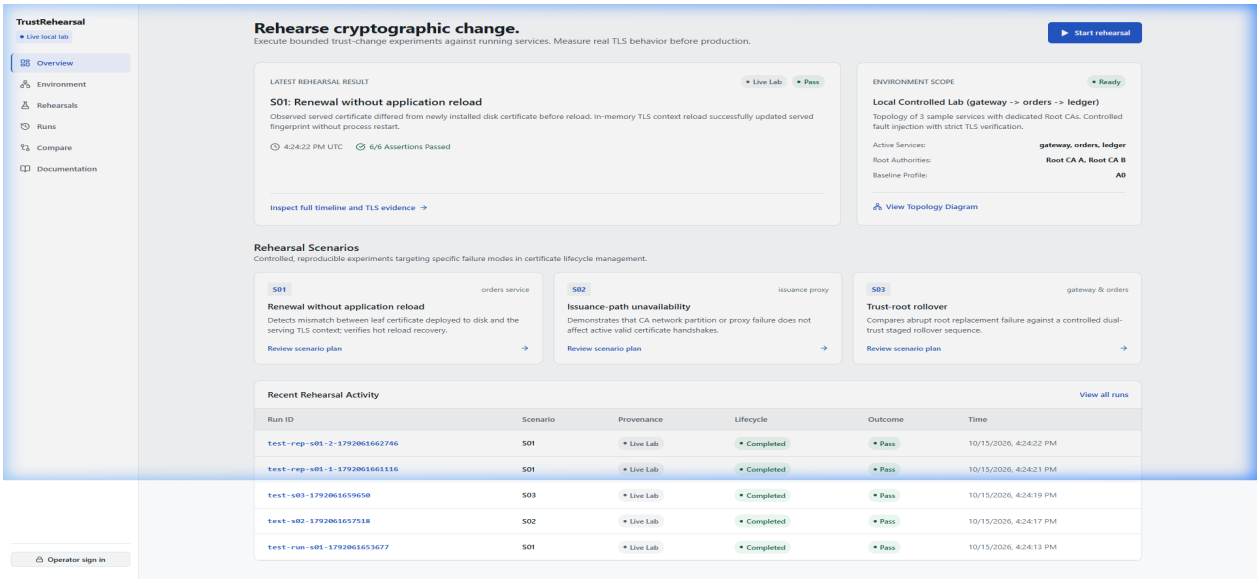


Figure 6: Application Overview: Mode badge, Hero trust path, Environment health summary, Scenario catalog cards, and Recent execution activity.

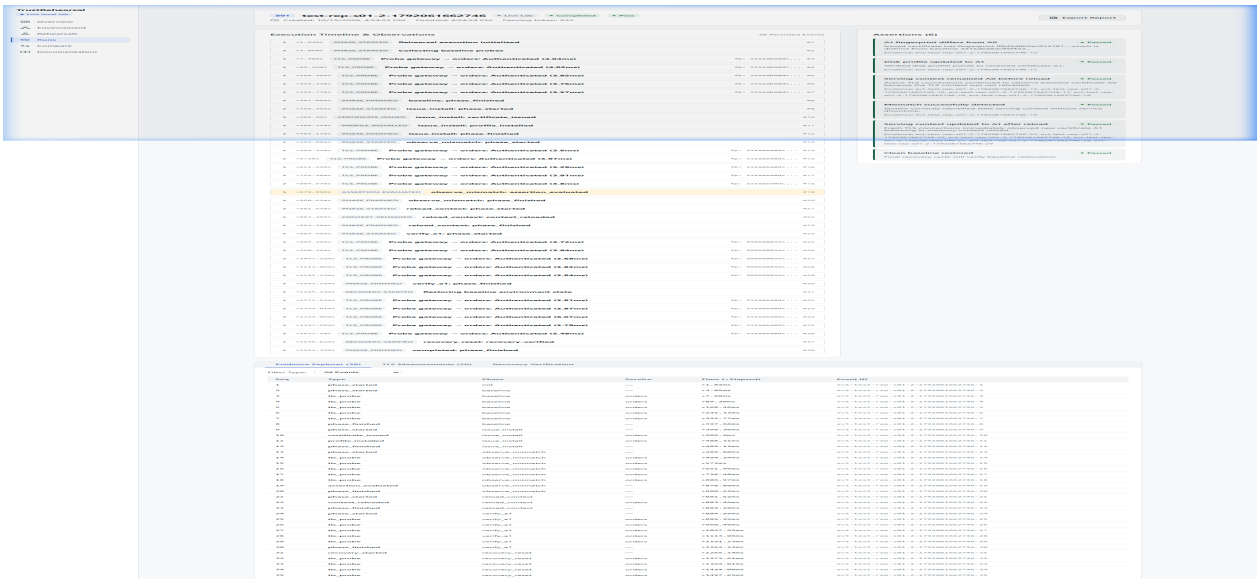


Figure 7: Run Detail & SSE Timeline: Interactive phase disclosures, peer leaf SHA-256 fingerprints, assertion results, and raw structured JSON inspection.

10. User Interface & Evidence Presentation (Continued)

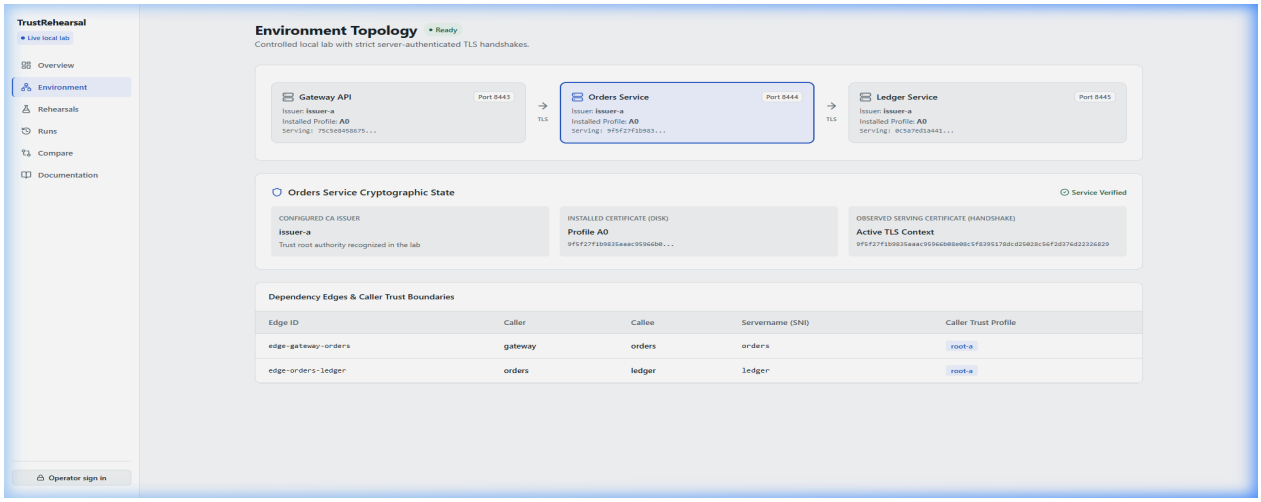


Figure 8: Environment Topology: Interactive SVG graph (gateway -> orders -> ledger) with node metadata and caller trust profiles.

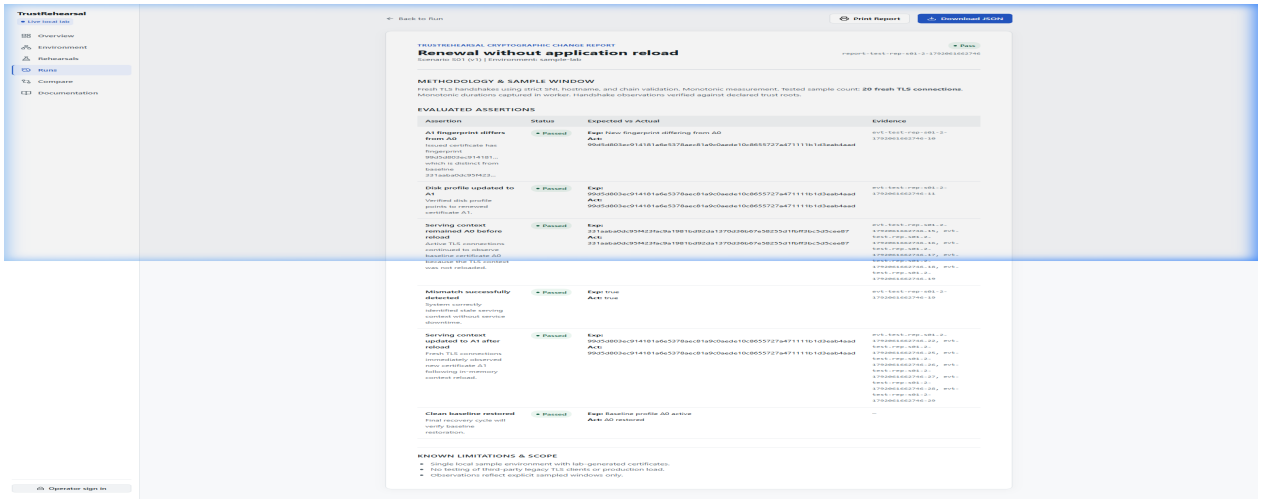


Figure 9: Canonical Report View: Validated methodology, observation windows, assertion outcomes, and clean printable A4 format.

10.2 Navigation & Information Hierarchy Analysis

The UI prioritizes rapid evidence discovery for technical reviewers. A visitor landing on the Overview grasps the core PKI trust path within 60 seconds, drills down into exact SHA-256 fingerprints within 2 minutes, and downloads a canonical report within 5 minutes. The layout strictly enforces that disabled controls (e.g. Start Rehearsal in public mode) are never rendered enabled to unauthenticated visitors.

11. Verification, Traceability & Test Results

Verification in TrustRehearsal is multi-tiered, spanning contract unit tests, live TLS socket proofs, API integration tests, scenario executions, and external review harnesses.

11.1 Requirement Traceability Matrix

Req ID	Functional Requirement Summary	Verifying Test Suite / Source File	Observed Test Evidence & Outcome
FR-001	Mode & Provenance Display	`security-and-viewer.test.ts`	Passed. Provenance persistent; mode badge rendered.
FR-002	Fresh TLS Handshake Probing	`tls-proof.test.ts`	Passed. Captured peer SHA-256 fingerprint; 0 pooling.
FR-003	S01 Renewal Without Reload	`runs-and-api.test.ts`	Passed. Detected mismatch; context reloaded.
FR-004	Durable Event Persistence	`recheck-16.cjs (R18)`	Passed. Write failure halts execution; returns null ID.
FR-005	Traceable Assertion References	`contracts.test.ts`	Passed. 100% assertions map to valid event IDs.
FR-006	Monotonic Fencing & Locks	`runs-and-api.test.ts`	Passed. Duplicate run rejected with HTTP 409 Conflict.
FR-007	Safe Baseline Reset	`reset-lab.ts`	Passed. Baseline restored and verified with live probe.
FR-008	S02 Issuance Proxy Fault	`scenarios-s02-s03.test.ts`	Passed. 3 HTTP 502s observed; data plane uninhibited.
FR-009	S03 Staged Root Rollover	`scenarios-s02-s03.test.ts`	Passed. Neg. control rejected; HTTP 200 on Root B.
FR-010	Graceful Run Cancellation	`runs-and-api.test.ts`	Passed. Ceased mutations; clean recovery within budget.
FR-011	Structured Evidence Filtering	`recheck-16.cjs (P1B)`	Passed. Query params (limit, afterSeq) sliced correctly.
FR-012	Canonical Report Export	`contracts.test.ts`	Passed. Validates ReportSchema; printable view clean.
FR-013	Run-to-Run Comparison	`ComparePage.tsx`	Passed. Mismatched scenario IDs rejected; diffs shown.
FR-014	Authenticated Operator Actions	`security-and-viewer.test.ts`	Passed. HTTP 401 on missing auth; 403 on invalid CSRF.
FR-015	Public Credential-Free Mode	`test-public-viewer.ts`	Passed. 9/9 checks passed; zero backend requests.
FR-016	Honest Measurement Integrity	`recheck-16.cjs (P1C)`	Passed. Empty snapshots render 0 samples; 503 is UNAVAIL.

11. Verification, Traceability & Test Results (Continued)

11.2 Independent Verification Results & Review Findings

Verification was conducted across three distinct test execution harnesses:

1. Automated Vitest Suite (Repository Unit & Integration Tests):

- **Result:** 27 of 27 automated tests passing across 6 suites (Execution time: 36.99s).
- **Suites:** contracts.test.ts, tls-proof.test.ts, runs-and-api.test.ts, scenarios-s02-s03.test.ts, security-and-viewer.test.ts, regressions-review16.test.ts.
- **Coverage:** Validates schema contracts, atomic certificate reloads, concurrent lease conflicts, S02/S03 scenario pipelines, CSRF denial, and regression checks R16, R17, and R18.

2. Independent Targeted Review Harness (recheck-16.cjs):

- **Result:** 8 of 8 targeted defect checks passed cleanly:
- *real-chain-and-management:* Baseline application path HTTP 200; missing management token rejected with HTTP 401.
- *s03-before-cleanup:* All 4 assertions pass; final application request HTTP 200; ledger confirmed on Root B.
- *issuance-fault-network:* HTTP renewal request traversed proxy and returned HTTP 502 Bad Gateway.
- *anonymous-issuer:* Anonymous CA POST /issue rejected with HTTP 401 Unauthorized.
- *event-persistence-failure:* Injected write failure properly halted run progression without false pass.
- *api-503-silent-fallback:* Backend HTTP 503 truthfully surfaces UNAVAILABLE rather than recorded fallback.
- *static-events-query:* Static query string parsing correctly handles limit and afterSeq.
- *static-report-missing-evidence:* Empty snapshots truthfully report 0 samples and not_run recovery.

3. External Public Viewer Verification Harness:

- **Result:** 117 assertions passed across the 9 recorded canonical runs.
- **Clarification on Review Evidence:** The 117-assertion harness is an external independent review script (final-viewer-check.cjs) executed during review auditing. It is not an internal bundled npm command, but serves as formal third-party audit evidence validating snapshot integrity, routing, and write denial.

11.3 Release Acceptance Gate Checklist

Verification Gate	Required Release Standard	Verification Result
Dependency Lockfile	Pinned dependencies; zero runtime installation research.	Passed. Pinned in package-lock.json.
TLS Proof Before UI	Real socket handshakes before dashboard implementation.	Passed. Verified in tls-proof.test.ts.
Scenario Reproducibility	All 3 scenarios reproduce across clean repetitions.	Passed. 3 clean runs per scenario (9 total).
Persistence Integrity	Event write failures halt execution without false pass.	Passed. Verified in regression test R18.
Security Boundaries	Anonymous mutations denied; zero private keys in exports.	Passed. Verified in security test suite.
Truthful Measurements	Empty snapshots report 0 samples; 503 is UNAVAILABLE.	Passed. Verified in independent review harness.

12. Technical Risks, Limitations & Future Roadmap

12.1 Known Limitations of Current Implementation

- **Single-Process Loopback Topology:** All sample services run within a single Node.js runtime on distinct loopback ports rather than separate container network namespaces. While this eliminates Docker socket risks, it does not evaluate inter-container bridge networking delays.
- **Server-Authenticated TLS Only:** The V1 release focuses exclusively on server-authenticated TLS. Mutual TLS (mTLS) client certificate verification is deferred to subsequent milestones.
- **Periodic Sample Windows:** Probe observations represent periodic sample bursts (e.g. 5 to 30 handshakes) rather than continuous high-concurrency production saturation.

12.2 Post-V1 Architecture Roadmap

- **Multi-Container Lab Topology (V2):** Containerize sample services into isolated OCI containers orchestrated via Docker Compose, utilizing dedicated loopback bridge networks without mounting host Docker sockets.
- **Post-Quantum Cryptography (PQC) Rehearsals:** Rehearse hybrid classical/PQC leaf and root transitions (e.g. ML-DSA / Dilithium) to evaluate TLS handshake packet fragmentation and legacy client rejection.
- **Automated GitOps CI/CD Integration:** Webhook integrations enabling pull-request previews to rehearse cryptographic changes in ephemeral staging environments prior to production merges.

13. Glossary of Terms & Authoritative References

13.1 Glossary of Terms

- **Certificate Authority (CA):** An authoritative entity issuing X.509 certificates that cryptographically bind public keys to identities.
- **Subject Alternative Name (SAN):** An X.509 extension specifying hostnames, IP addresses, or URIs secured by the certificate.
- **Server Name Indication (SNI):** A TLS client extension indicating the target server hostname at the start of the handshake.
- **SHA-256 Fingerprint:** The 64-character hexadecimal SHA-256 digest of a certificate's raw DER-encoded bytes.
- **Dual Trust:** A transitional configuration where client trust bundles include both expiring and target Root CAs.
- **Fencing Token:** A monotonic integer preventing stale or partitioned workers from applying outdated state mutations.
- **Negative Control:** An experiment step intentionally designed to trigger an authentication failure, proving security checks are active.

13.2 Authoritative Standards & References

1. **RFC 5280:** Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile.
2. **RFC 8446:** The Transport Layer Security (TLS) Protocol Version 1.3.
3. **RFC 6125:** Representation and Verification of Domain-Based Application Service Identity within Internet PKI.
4. **NIST SP 800-52 Rev. 2:** Guidelines for the Selection, Configuration, and Use of TLS Implementations.
5. **Project Repository:** https://github.com/SubhodeepSamanta/TrustRehearsal.git

---

TrustRehearsal Engineering Specification · Version 1.0 · Authored by Subhodeep Samanta